Compte Rendu final de Projet de Recherche

Pierre Baumann, Ivan Mulot-Radojcic, Damien Assire

6 mai 2024

Résumé

Abstract

Table des matières

1 Introduction **1**

2 Génération du graphe **1**

3 Génération du graphe **1**

4 Calcul de la mesure **2**
4.1 Ancien algorithme 2
4.2 Nouvel algorithme 2
4.2.1 Création d'une structure de données adaptée 2
4.2.2 Algorithme de calcul de la mesure 3
4.2.3 Optimisations faites 3
4.2.3.1 Calcul de la distance entre le noeud de départ et tous les autres noeuds 4
4.2.3.2 Prise en compte de tous les sous-chemins 4
4.2.3.3 Prédiction des chemins avec le plus d'étapes 4
4.2.3.4 Regarder ses voisins 4
4.2.3.5 Se rapporter à un parcours de graphe classique quand c'est possible . 5
4.2.3.6 Calcul du non-attqué 5
4.2.4 Complexité de l'algorithme 5

5 Résultats et interprétation **5**

6 Perspectives et ouvertures **5**

7 Conclusion **5**

sections

1 Introduction

Terminol

Introduct

2 Génération du graphe

Générati du graphe

3 Génération du graphe

Générati du graphe

4 Calcul de la mesure

4.1 Ancien algorithme

Ancien algorithme

4.2 Nouvel algorithme

Le problème des plus courts chemins est déjà assez complexe dans un graphe classique, mais il devient encore plus difficile dans un graphe temporel, où les arêtes peuvent être bloquées à certains moments. Nous sommes d'abord partis sur une adaptation de l'algorithme de Floyd-Warshall, car nous savons que cet algorithme marche tout type de graphe, notamment des graphes étranges comme les ceux avec des arêtes à poids négatifs.

Le problème des calculs de chemin dans les graphes temporels

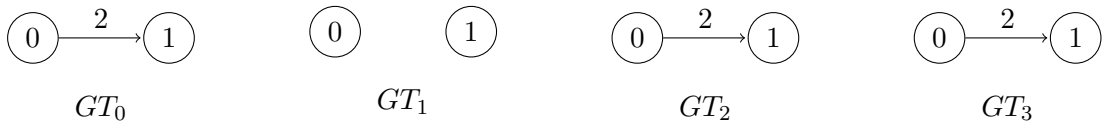
TemporalFloydWarshall(graphe temporel GT, DistanceMatrixes dma)

```
InitTFW(GT, dma)
for time in {dma.NumMatrixes .. 0} do
    for k in {0 .. |GT.V| } do
        for u in {0 .. |GT.V| } do
            for v in {0 .. |GT.V| } do
                distUV ← dma.matrix[t_i][u][v]
                distUK ← dma.matrix[t_i][u][k]
                distKV ← dma.matrix[t_i + distUK][k][v]
                if distUV > distUK + distKV then
                    dma.matrix[t_i][u][v] ← distUK + distKV
                end if
            end for
        end for
    end for
end for
```

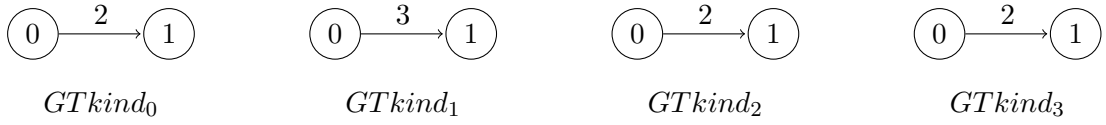
4.2.1 Création d'une structure de données adaptée

Le problème des autres algorithmes classiques de graphes est qu'ils ne peuvent pas voir dans le futur, et donc ne peuvent pas prendre en compte les liens bloqués puis débloqués. Cependant, nous avons pensé à créer des liens fantômes qui remplacent les liens supprimés, équivalents à la distance du lien d'origine plus l'attente nécessaire pour que le lien soit à nouveau disponible. Cela va permettre de ne pas avoir besoin de regarder dans le futur pour calculer les chemins, tout en calculant quand même la vraie distance minimale, car le cas où attendre est bénéfique a été pris en compte par le lien fantôme. Ainsi, nous avons pu rendre beaucoup plus de parcours de graphes adaptables temporellement, notamment Dijkstra, que nous avons utilisé pour notre implémentation.

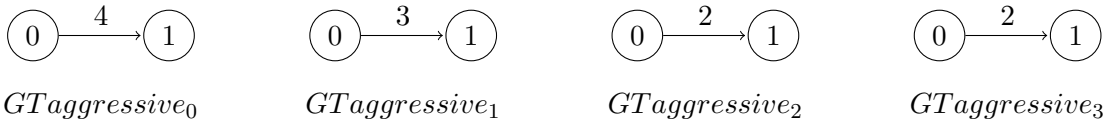
Mettre ca en vertical et rendre ça moins moche et finir exemple



Lien de base



Stratégie gentille



$GTaggressive_0$ $GTaggressive_1$ $GTaggressive_2$ $GTaggressive_3$

Stratégie agressive

4.2.2 Algorithme de calcul de la mesure

Comme dit précédemment, nous avons adapté l'algorithme de Dijkstra pour les graphes temporels. Avec la nouvelle structure de données, la seule différence avec le dijkstra classique est qu'au lieu de regarder la distance avec le noeud actuel, on regarde la distance avec le noeud voisin au temps déjà parcouru.

TemporalDijkstra(graphe temporel GT, Noeud depart, Noeud arrivee) -> Distance

```
NoeudVisited ← ∅
NoeudsATraiter ← {depart}
while nodesToVisit ≠ {} do
    Noeud currentNode ← nodesToVisit.pop()
    if currentNode is arrivee then
        return visited[currentNode]
    end if
    visited[currentNode] ← distance
    PhantomEdges neighbours ← GT.getNeighbours(currentNode)
    for voisin in neighbours do
        distance dist ← visited[currentNode] + voisin[visited[currentNode]]
        if voisin not in visited or visited[voisin] > dist then
            visited[voisin] ← dist
            nodesToVisit.push(voisin)
        end if
    end for
end while
```

---

[1]

A noter que cet algorithme reste correct pour tout type de graphe temporel (discret) ou flot de liens (continu), multi-graphe [2] ou non, orienté ou non, tant qu'il vérifie ces 2 propriétés : Pour tout noeud u, v, ayant pour poids au cours du temps $p_{u,v}(t)$,

— $p_{u,v}(t) > 0 \ \forall t \in \mathbb{R}$, car l'algorithme de Dijkstra ne peut pas travailler avec des poids négatifs.

— $\frac{p_{u,v}(t)}{p_{u,v}(t')} \geq -1 \ \forall t, t' \in \mathbb{R} | t > t'$, car notre algorithme suppose qu'une attente n'est jamais bénéfique.

Mettre au propre temporal dijkstra

4.2.3 Optimisations faites

L'algorithme de Dijkstra est donc mis en place et adapté pour les graphes temporels, mais il reste encore des optimisations à faire. En effet, l'algorithme de Dijkstra est souvent utilisé pour calculer le plus court chemin entre deux noeuds, mais il calcule en réalité beaucoup plus de plus courts chemins. Donc nous avons modifié l'algorithme pour qu'il écrive directement dans la matrice de distances, puis le but va être d'écrire le plus de distances possibles en une seule fois.

Nous avons prit parti d'optimiser pour le temps et non la mémoire, le projet étant fait pour tourner sur des supercalculateurs avec des centaines de Go de RAM. Ainsi que le temps global de plusieurs

1. L'algorithme est simplifié et renvoie une distance et non un chemin, mais il est facile de le modifier pour qu'il renvoie un chemin.
2. Graphe ayant plusieurs liens entre 2 noeuds

attaques plutôt que le temps d'une seule attaque, le but du projet étant de regarder une évolution de l'efficacité des attaques.

4.2.3.1 Calcul de la distance entre le noeud de départ et tous les autres noeuds

Pour connaître la distance entre le noeud de départ et le noeud d'arrivée, Dijkstra va explorer beaucoup de noeuds candidats. Donc, pour chaque noeud candidat à une distance d du noeud de départ qu'il sort du tas, on va noter cette distance dans la matrice de distances.

4.2.3.2 Prise en compte de tous les sous-chemins

Soit un chemin calculé 0->1->2->3, on a un chemin trivial à noter entre 0 et 3, mais on a aussi beaucoup de sous-chemins qui sont calculés. Ici, on a les chemins 0->1->2, 1->2->3, 0->1, 1->2, 2->3.

Valeur à l'index 2 : two

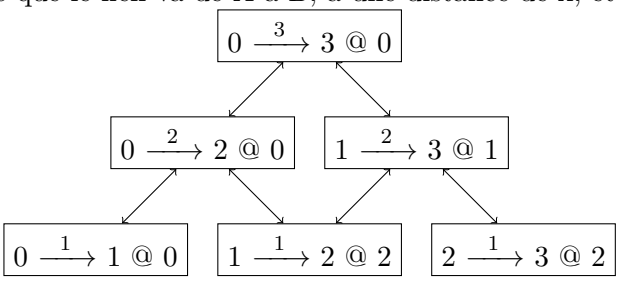
$0 \xrightarrow{1,\ 1,\ 1[0]} 1 \xrightarrow{1,\ 1,\ 1[2\text{-}2]} 0,\ 1,\ 2,\ 3[2]\ \ 2 \xrightarrow{1,\ 1,\ 1[3\text{-}2]} 0,\ 1,\ 2,\ 3[3]\ \ 3 \xrightarrow{1,\ 1,\ 1[4\text{-}2]} 0,\ 1,\ 2,\ 3[4]$

$\xrightarrow{1}\ \xrightarrow{1}\ \xrightarrow{1}$ Et en général, pour un chemin de taille n, on a $n(n-1)/2$ sous-chemins qu'on peut noter dans la matrice de distances.

Nous avons donc créer une structure de données qui va permettre de rapidement stocker tous les sous-chemins à partir des pas faits dans le chemin. Cette structure ressemble à un arbre de segments, mais en structure pyramidale.

Les éléments $0 \xrightarrow{1} 1$, $1 \xrightarrow{1} 2$, $2 \xrightarrow{1} 3$ sont les briques de base de la pyramide, qui sont l'équivalent des feuilles dans un arbre. Dans notre pyramide, chaque élément va stocker le noeud de départ, le noeud d'arrivée, la distance entre ces deux noeuds, et le temps nécessaire pour arriver au noeud de départ. Ensuite, pour chaque étage de la pyramide, on va regarder notre fils gauche et notre fils droit, ainsi que notre petit-fils commun. Le noeud de départ sera celui du fils gauche, le noeud d'arrivée celui du fils droit, la distance sera la somme des distances des deux fils moins la distance du petit-fils (partie commune comptée deux fois sinon), et le temps nécessaire pour arriver au noeud de départ sera celui du fils gauche. S'il n'y a pas de petit-fils, alors il n'y a pas de distance commune, et donc la distance sera la somme des distances des deux fils.

Pour l'exemple précédent, notre pyramide ressemblerait à ça : Avec la notation $A \xrightarrow{x} B$ @ t pour dire que le lien va de A à B, a une distance de x, et qu'il faut t ticks pour arriver à A.



4.2.3.3 Prédiction des chemins avec le plus d'étapes

Cette optimisation va de pair avec la précédente, en faisant des benchmarks, nous avons remarqué que parcourir la boucle j à l'envers permettait de gagner du temps. On suppose que c'est dû au fait que les noeuds avec des indices éloignés soient aussi éloignés dans le graphe, donc que le chemin entre ces deux noeuds ai plus d'étapes, et que donc on va écrire plus de sous-chemins dans la matrice de distances qui seraint écrasés plus tard si on avait parcouru la boucle dans le sens normal.

4.2.3.4 Regarder ses voisins

La distance de u à v est le minimum entre la distance de u à k plus la distance de k à v pour chaque noeud k tel que l'arête $u->k$ et l'arête $k->v$ existent. Donc si nous connaissons toutes ces distances $u->k$, alors nous pouvons calculer la distance $u->v$ sans avoir besoin de reparcourir le graphe.

Nous avons utilisé cette propriété pour optimiser notre algorithme. Tout d'abord, nous créons le graphe inverse, qui pour chaque arête $u->v$ du graphe original va créer une arête $v->u$.

Ensuite, pour avoir la distance entre u et v, nous cherchons tous les noeuds k tels que l'arête $k->v$ existe grâce au graphe inverse. Pour chaque noeud k, si la distance est calculée, alors nous pouvons directement la récupérer, sinon nous la calculons. Ensuite, nous prenons le minimum de toutes ces distances ($u->k+k->v$) pour avoir la distance $u->v$. C'est très utile pour les noeuds qu'on ne peut atteindre que par un seul autre noeud, car ça évite de refaire un parcours avec juste un noeud en plus.

4.2.3.5 Se rapporter à un parcours de graphe classique quand c'est possible

Certaines stratégies d'attaque sont statiques, et ne dépendent pas du temps. Cela implique que toutes les matrices de distances sont les mêmes, et donc que les calculer pour chaque matrice est inutile. Ainsi, on a divisé le format d'attaque en deux, un format statique et un format dynamique, et lorsque l'algorithme détecte que l'attaque est statique, il va se rapporter à un parcours de graphe classique avec une seule matrice de distances, puis calculer l'efficacité de l'attaque comme s'il y en avait plusieurs.

Ref vers attaques

4.2.3.6 Calcul du non-attqué

Nous nous sommes rendus compte que les matrices de distances plus loin dans le temps étaient beaucoup plus vides que les premières, dû à la nature de la mesure, qui va ignoré tous les chemins impossible à compléter dans notre lapse de temps. Par exemple, si notre tick max est de 100, notre tick courant est de 70, alors on ne peut pas calculer un chemin de plus de 30 ticks. Surtout que ne pas pouvoir le calculer signifie qu'on ne peut pas écrire plusieures valeurs dans la matrice de distances car il n'y a pas de chemin complet. Donc, on commence par calculer les distances sur le graphe non-attaqué, grâce à la méthode précédante, puis on dit au programme de ne calculer des chemins qui ont une chance de finir dans le temps imparti.

4.2.4 Complexité de l'algorithme

Donc même si la compléxité pire cas de faire des Dijkstra pour chaque noeud est en $O(n^3 \log n*t)$, la complexité amortie est bien meilleure. La calculer réellement serait extrêmement complexe et dépend beaucoup de la topologie du graphe, mais sur des villes réelles, nous avons mesurer l'évolution du temps de calcul en fonction du nombre de noeuds, et nous avons obtenu cette courbe : qui peut être modélisée par une fonction en $O(???????)$.

Courbe de la complexité

5 Résultats et interprétation

Calcul de la mesure

Résultats et interprétation

6 Perspectives et ouvertures

Perspecti et ouvertures

7 Conclusion

Terminol

Conclusi

Références